CPU power limits: experimental design

INF-2204 - 2

Group and project

Payam Nami, pna020@uit.no
William-Nima Felahi, wfe002@uit.no

We will use the desktop from Task 1 to test what happens when the CPU is allowed less power. The questions are still about performance, differences between workloads, energy efficiency and temperature. This report describes how we plan to measure them. No CPU benchmark results have been collected for this project yet.

Our expectation is conditional: a higher limit should help only when the workload actually reaches the power limit. Otherwise, changing the BIOS number may do very little. We want to check that rather than assume every program behaves like a full-core render.

1. Hardware environment

CPU	AMD Ryzen 7 9850X3D: Zen 5, 8 cores / 16 threads, 96 MB L3 cache, 120 W default TDP, 95 C maximum operating temperature [1].
Motherboard	ASRock X870E Taichi OCF, AM5. Current BIOS: 4.41.
Memory	32 GB V-Color DDR5, 2 x 16 GB, TMXFL1680838KWK. Windows reports 8000 MT/s configured data rate.
GPU	NVIDIA GeForce RTX 5080. Same driver and default GPU configuration throughout; no GPU tuning.
Storage	Samsung 9100 PRO 2 TB and Lexar NM790 2 TB. Planned input/output location: the Samsung SSD, with enough free space for all runs.
Physical setup	One unchanged cooler, case, PSU and fan curve. Their exact models still need to be recorded before the pilot; we cannot identify them reliably from Windows.

These are the available parts, not a claim that the test setup is already validated. AMD lists DDR5-5600 support for two DIMMs [1], so the reported 8000 MT/s profile needs a stability check. We will record timings and memory-controller settings, then keep the same validated profile for every PPT setting.

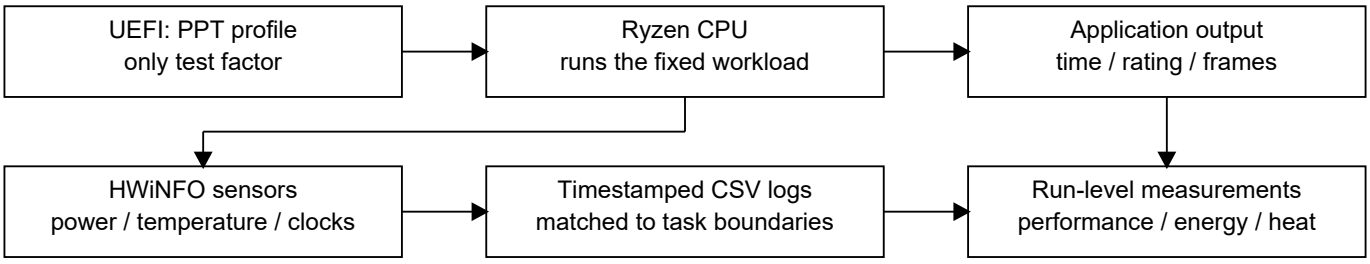


Figure 1. Planned measurement path on one PC. PPT changes the CPU's power allowance; application outputs and sensor logs are joined using the same run boundaries. Arrows show data or control flow, not measured results. Diagram by the group.

2. Software environment

The OS will be Windows 11 Pro. We plan to use Windows Balanced for the whole campaign and keep the same chipset and GPU drivers, security settings and hardware-accelerated GPU scheduling setting. We will save their configuration before testing. Updates, downloads, overlays and other monitoring programs must not run during a recorded test.

Tools	HWiNFO64 8.34 and 7-Zip 25.01 x64 are currently available. Planned additions: Blender 4.5 LTS, HandBrakeCLI and PresentMon Console. The exact patch versions will be recorded and frozen after the pilot.
Logging	HWiNFO CSV logging at a planned 1000 ms interval [3]. Log CPU package power, CPU (Tctl/Tdie) temperature, effective clocks, PPT/TDC/EDC usage and limit flags. Also log GPU use during the game.
Frame capture	PresentMon targets only the game process and main swap chain. Use its legacy CSV mode for MsBetweenPresents and absolute timestamps; keep the mode identical in every capture [9].

No simulator or custom benchmark code is needed for this design. We will use the tools' existing commands and saved presets. The final commands, presets and versions will be kept with the data; analysis can be done in a spreadsheet using the formulas in Section 5.

3. Workloads

CPU rendering

Blender will render one fixed Classroom frame with Cycles on the CPU only, using 16 threads [4, 5]. The planned settings are frame 1, 1920 x 1080, 128 samples, fixed seed, adaptive sampling off and denoising off. We will save the configured scene. This refines Task 1's Blender test into a fixed render, so each run finishes the same work rather than relying on a time-limited score.

Compression and video encoding

For 7-Zip we plan `7z b 10 -mmt16 -md25`: ten internal iterations, 16 requested benchmark threads and an upper dictionary size of 2^25 bytes (32 MiB). The command can test several dictionary sizes; we will retain the complete output and compare the same final average compression and decompression ratings separately [6]. These are benchmark MIPS ratings, not hardware instruction counters.

HandBrake will encode the first 180 seconds of one fixed Tears of Steel source. We plan software x265, 8-bit output, medium encoder preset, RF 22 and constant 24 FPS. Keep the source dimensions, disable cropping, filters, audio, subtitles and hardware decoding, and write an MKV file. This is a sustained CPU workload; NVENC or another GPU encoder would answer a different question [7, 8].

One game benchmark

The planned choice is Cyberpunk 2077's built-in benchmark [10], subject to confirming access before the pilot. Start at 1920 x 1080, Low preset, low crowd density, and turn off ray tracing, upscaling, frame generation, VSync and FPS caps. We will check 720p during the pilot too. If reducing resolution still improves FPS beyond repeat-run variation, 1080p is not a clean CPU-limited test. We will freeze a suitable setting before collecting data, or report the remaining GPU limit honestly.

4. Datasets and models

The inputs are the official Classroom scene (CC0) [5], the 4K Tears of Steel source described in HandBrake's documentation (3840 x 1714, 24 FPS) [8], 7-Zip's internally generated LZMA test data [6], and the game's fixed benchmark sequence. The scene and video provide repeatable real application inputs; 7-Zip needs no external archive. Save source URLs, licences, file sizes and SHA-256 hashes before testing, plus the edited scene and encode preset. No simulation model is planned.

5. Evaluation and performance metrics

Each measurement belongs to one completed run at one PPT profile. The stock profile is the comparison point. We will keep the configured PPT in watts separate from measured package power: the cap is a ceiling, not the power the CPU necessarily uses.

Performance and timing

For Blender and HandBrake, task time T is wall-clock seconds from process launch to successful exit. This includes input loading and output writing, consistently across profiles. Lower time is better; for percentage comparisons we use performance S = 1/T. For 7-Zip, S is its reported compression or decompression rating. Gaming uses average FPS, calculated as 1000 divided by the mean captured frame interval in milliseconds.

For the game we will use MsBetweenPresents from the same process and swap chain [9]. Remove menu/loading intervals and any interval crossing a scene boundary. Let N be the retained interval count and T their total duration; match power integration to those present timestamps. Report median, P95 and P99 frame time. Our 1% low FPS is 1000 divided by the mean of the slowest ceil(0.01 x N) intervals. This measures application frame pacing, not input latency or physical display timing.

Package energy, power and efficiency

HWiNFO provides the power samples; it is not an external electricity meter [3]. We will match its timestamps to the task interval, interpolate only the two boundary values and use the actual spacing between samples:

$$E_{pkg}\ (J) \approx \text{sum of } [(P_i + P_{i+1})/2] \times (t_{i+1} - t_i)$$
$$\text{Mean package power } (W) = E_{pkg} / T$$

P is logged CPU package power in watts and t is time in seconds. We will not calculate energy as PPT x time. The main value includes the package's idle/background contribution during the task, without subtracting an idle estimate. It excludes GPU, PSU and wall-socket energy. Software energy measurement has limitations [11]; the confidence intervals below will not tell us the sensor's absolute accuracy.

For the two fixed jobs, report joules per completed render or encode; performance per watt is $(1/T)/(E_{pkg}/T) = 1/E_{pkg}$. For 7-Zip, use each rating divided by mean package watts over the full benchmark, not a separate power reading for each phase. Equal iteration counts may not mean equal bytes processed. For gaming, report FPS/W and E_{pkg}/N in J/frame over the matched interval. FPS/W is then the inverse of J/frame.

Comparisons, heat and uncertainty

Within each workload, normalised performance is $100 \times S/S_{stock}$. Between adjacent numeric PPT limits, report performance change as $100 \times (S_{next}/S_{current} - 1)$. For fixed jobs, report energy change the same way using E. For gaming, use J/frame; for 7-Zip, compare rating/W. A gain-per-extra-energy ratio is only useful when the energy increase is positive. If it is zero or negative, show the two changes instead of forcing a ratio.

Report mean and peak CPU temperature in C, average effective clock and the proportion of samples with PPT usage at least 95%. Check TDC, EDC and thermal flags too: a flat performance curve could be another limit, not proof that PPT never matters [2]. For longer renders and encodes, also report mean temperature over the final 60 seconds when the job lasts at least three minutes.

Use at least five valid independent runs per condition. Show raw run points, mean, sample standard deviation and approximate 95% confidence intervals: mean +/- $t_{0.975,n-1}$ x s/sqrt(n) [12]. These intervals assume roughly independent, reasonably behaved run values; five runs give limited evidence. Frames and one-second sensor samples are not extra repetitions. Planned plots put PPT (W) on the horizontal axis and performance (% of stock), energy (J/job or J/frame), efficiency or temperature (C) on labelled vertical axes. Stock is a separately labelled point until its actual PPT is known.

6. Experimental protocols

Power profiles and controlled settings

The main factor is PPT, the socket power allowance. TDC limits sustained current and EDC peak current [2]. Start with 65, 88, 105, 125 and 150 W plus stock/default. AMD's 120 W TDP is not the stock PPT value [1, 2]. Save the current configuration, then establish a stock CPU baseline with no manual PPT or motherboard enhancement preset. Verify the effective Auto/default limits and record screenshots of PPT, TDC, EDC and temperature limits before testing.

In the manual profiles, keep stock baseline TDC/EDC, Curve Optimizer at zero, boost override at zero, default scalar and thermal protection enabled. SMT stays on; core count, memory profile/timings, GPU settings, fan curve and Windows plan stay fixed. If entering PPT requires manual PBO, document that and explicitly match the other controls [2]. Do not use a motherboard 'unlimited' preset. If a proposed limit exceeds the verified stock ceiling, revise the set to stay at or below stock before the main campaign; remove duplicate limits.

Pilot and run order

Before recording main data, check that each profile is stable and really applies the intended limit. Confirm the memory profile, correct outputs and timestamp alignment. Compare monitoring on/off in three paired pilot runs of the render and game. If the difference exceeds normal pilot variation, simplify the sensor list or slow logging, then freeze one interval. For rendering, raise the sample count once during the pilot if the stock job is shorter than three minutes; save that final scene before any main comparisons.

Use five blocks initially. In each block, randomly order all six planned power profiles and save that order. Reboot after a UEFI profile change and verify the profile in HWiNFO. Run all four workloads once per profile, rotating their order between blocks. This gives 120 recorded runs if all six profiles are retained. If pilot variation (standard deviation / mean) exceeds 3% for a workload, plan ten blocks for that workload at every profile, rather than adding repeats only to an inconvenient result.

At each profile, give each workload one unrecorded warm-up. Before every measured run, wait at least two minutes and require CPU idle temperature within 2 C of the session's idle reference for 60 seconds. Record room temperature with the same thermometer; pause if it drifts more than 2 C from the session start. If the PC has not settled after ten minutes, check why. Close background applications and warm the input cache consistently rather than mix cold and warm starts.

Protocol for each workload

Blender	Run the saved CPU-only scene in background mode. Capture launch/exit times, render log and output PNG. Accept only a successful render with the expected dimensions and sample count.
7-Zip	Run the fixed command from Section 3. Save complete stdout, launch/exit times and both average ratings. Treat its ten internal iterations as one trial; reject a reported data error, not a low rating.
HandBrake	Load the frozen input and preset; run one CPU encode to a unique output file. Save the activity log, launch/exit times and output duration/frame count. Verify the same 180-second segment, dimensions and encoder each time.
Game	Warm the same benchmark sequence, then capture only the repeatable scene. Save a settings screenshot, built-in result and PresentMon CSV. Define the scene start/end markers in the pilot and use them for every run; no manual play or changing save files.

Start HWiNFO 30 seconds before each run and stop 30 seconds after it. For process-based jobs, use Windows process timing with start/end timestamps on the same clock as the CSV; for the game use absolute frame timestamps. Align boundaries within one logging interval and record that timing uncertainty. Flag gaps above twice the fixed interval, missing power values, crashes or visible background interruptions. Keep the failed run and reason, then repeat that condition. A large value alone is not a reason to delete it.

7. Timeline for the rest of the semester

The dates below are a plan from 1 October onward, not a record of finished work. They continue the Task 1 schedule; the final conference date still needs confirmation.

1-11 October	Apply design feedback, record missing physical parts, confirm game access, run pilots and freeze settings, versions and inputs.
12 Oct-1 Nov	Collect the randomised blocks. Check logs and outputs after each session and back them up.
2-8 November	Review failures, repeat documented invalid runs and finish the planned repetitions.
9-15 November	Calculate metrics and uncertainty; make graphs and compare the four workloads.
16-22 November	Revise the report, check claims against raw data and prepare the presentation.
23 Nov, tentative	Project conference, following the provisional date used in Task 1.

8. Discussion and data management

Keep one folder per run, such as block01_ppt088_blender, containing raw CSVs, tool output and a short run record. Record profile, order, start/end times, ambient temperature, input hash, versions and any interruption. Keep configurations and pilot data separately. Raw files stay unchanged; derived tables must identify their source runs. Back up the folder after every session, with both participants able to access it. Share presets and permitted inputs, not game installation files.

The main risks are sensor error, background activity, unstable memory and a game that remains GPU-limited. A fixed logging setup helps comparison but does not calibrate the power sensor. The 8000 MT/s memory setting also matters: if it is unstable, we will choose a stable profile before collection and use it everywhere, not switch halfway through. Keep temperature protection on and stop a test that crashes or reports errors.

We are not sure yet whether the game will separate the PPT profiles clearly. It may stay below the cap even at low settings. That would still describe this workload, but it would not show that reducing PPT is harmless in every game. Likewise, one scene and one video cannot represent all rendering or encoding. We will limit the conclusion to this PC and these settings, rather than name one power limit as best for everyone.

References

Tool and hardware documentation accessed 1 October 2026. References support the technical definitions; the specific protocol choices are our planned design.

[1] AMD. Ryzen 7 9850X3D specifications. Processor specifications.

[2] AMD. Ryzen Master User Guide, CPU controls. PPT, TDC and EDC controls.

[3] HWiNFO. Software capabilities and sensor logging. Monitoring and logging.

[4] Blender Foundation. Blender 4.5 LTS Manual, command-line arguments. CPU rendering options.

[5] Blender Foundation. Demo Files: Classroom, by Christophe Seux, CC0. Scene source.

[6] Igor Pavlov. 7-Zip 25.01 User Manual, b (Benchmark) command. Bundled 7-zip.chm. Official distribution.

[7] HandBrake. Command line reference. Encode options.

[8] HandBrake. Performance documentation, including Tears of Steel test source. CPU encoding and source video.

[9] Intel / GameTechDev. PresentMon Console documentation. Frame CSV definitions.

[10] CD PROJEKT RED. Patch 1.5 notes: PC benchmark mode. Built-in benchmark.

[11] G. Raffin and D. Trystram. Dissecting the Software-Based Measurement of CPU Energy Consumption: A Comparative Analysis. IEEE TPDS 36(1), 96-107, 2025. Research paper.

[12] NIST/SEMATECH. e-Handbook of Statistical Methods, confidence limits for the mean. Confidence interval definition.